

# Private Network Service Platform

Computer Networks course project - Team 1 - Phase 1: Build & Observe  
Architecture Document - October 2026

## 1. Overview

A client opens **https://app.team1.test**. The name is resolved by our own DNS server (Mac 1), the HTTPS connection terminates at an nginx edge (Mac 2), and nginx load-balances each request to one of two backends (Mac 3 and Mac 4). Everything runs locally on four Macs on one private Wi-Fi LAN, with no cloud services. The application is deliberately simple; the network is the project.

**Clients talk to only two machines: Mac 1 for the name, Mac 2 for the page**

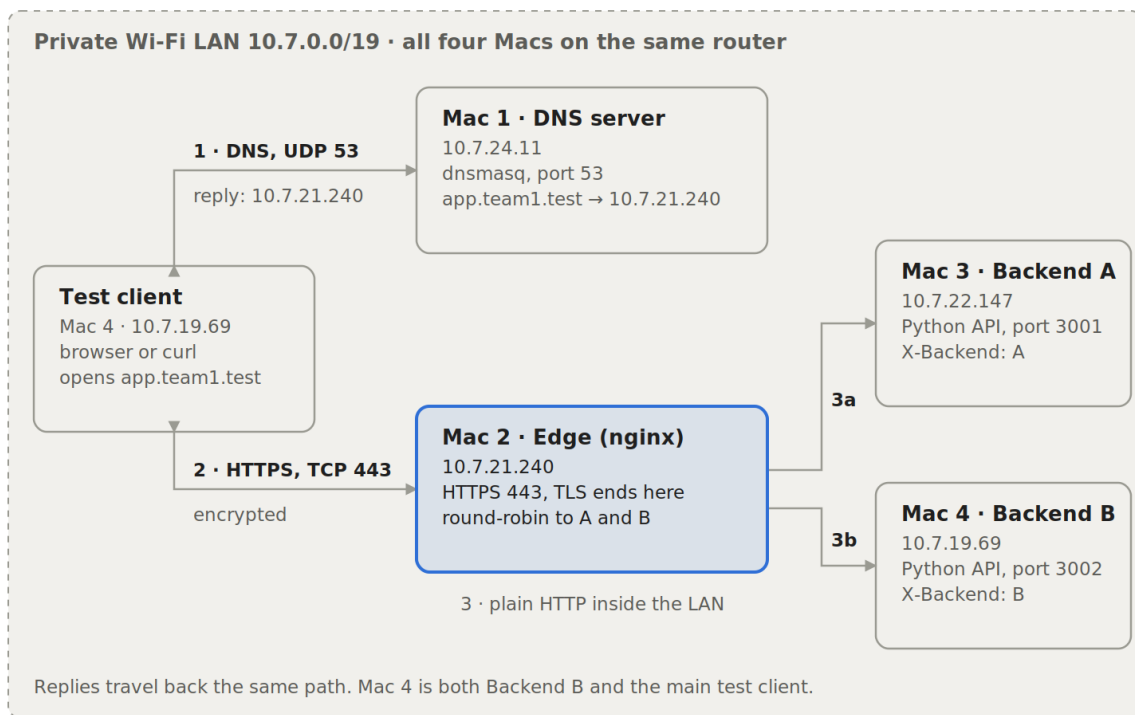


Figure 1 - Network topology

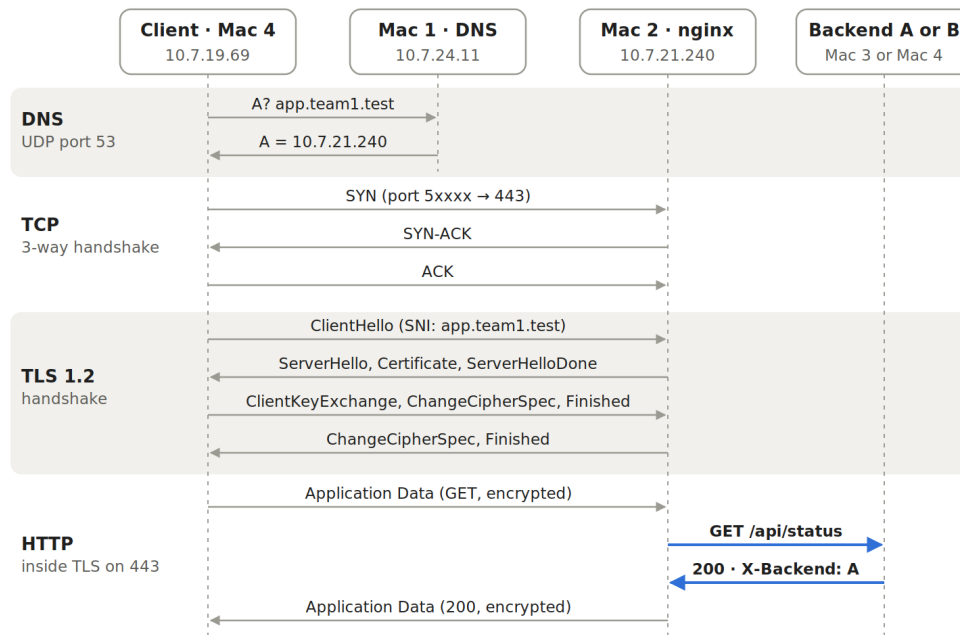
## 2. Machine roles and cloud equivalents

| Mac   | Owner    | IP address  | Role                                                | Runs                                       | Cloud equivalent            |
|-------|----------|-------------|-----------------------------------------------------|--------------------------------------------|-----------------------------|
| Mac 1 | Harshita | 10.7.24.11  | Private DNS server + test client                    | dnsmasq (UDP/TCP 53), dig, curl            | AWS Route 53                |
| Mac 2 | Yashi    | 10.7.21.240 | Edge reverse proxy + load balancer, TLS termination | nginx (TCP 80 / 443), server certificate   | AWS ALB / GCP LB / CDN edge |
| Mac 3 | Manan    | 10.7.22.147 | Backend A                                           | Python REST API, TCP 3001                  | EC2 application server      |
| Mac 4 | Akhil    | 10.7.19.69  | Backend B + main test client                        | Python REST API, TCP 3002, curl, Wireshark | EC2 application server      |

Network 10.7.0.0/19 (mask 255.255.224.0), gateway 10.7.0.1, interface en0 on every Mac. Private domain **team1.test**: both `app.team1.test` and `api.team1.test` are A records pointing at Mac 2 with a 30 second TTL.

### 3. Request flow

Every request: DNS, then TCP, then TLS, then HTTP, readable only behind Mac 2



Highlighted: the only hop where Wireshark can read HTTP (Mac 2 to backend, port 3001 or 3002).

Figure 2 - Request flow from client to backend

- DNS** - Mac 4 asks Mac 1 for app.team1.test over UDP (ephemeral port -> 53). Mac 1 answers 10.7.21.240 with TTL 30.
- TCP** - Mac 4 opens a connection to 10.7.21.240:443 with the SYN, SYN-ACK, ACK handshake.
- TLS** - ClientHello (SNI app.team1.test) -> ServerHello -> Certificate signed by Team1 Local Root CA -> key exchange -> Finished. Everything after this is encrypted.
- HTTP** - the GET request travels inside TLS (HTTP/2 by default, HTTP/1.1 also supported).
- Load balancing** - nginx decrypts the request and forwards it as plain HTTP to Backend A (10.7.22.147:3001) or Backend B (10.7.19.69:3002) in round-robin order.
- Response** - the backend replies with an X-Backend: A or B header; nginx adds X-Edge: mac2-nginx and returns it over the same TLS connection.

The client never learns the backend addresses: DNS only ever returns Mac 2, and nginx chooses the backend.

### 4. Protocol layer mapping

| Protocol         | TCP/IP layer                      | OSI layer                  | Where it is shown                      |
|------------------|-----------------------------------|----------------------------|----------------------------------------|
| DNS              | Application                       | 7 Application              | dig, Wireshark dns filter, dnsmasq log |
| HTTP/1.1, HTTP/2 | Application                       | 7 Application              | curl -v, browser DevTools              |
| TLS 1.2 / 1.3    | Between Application and Transport | 5-6 Session / Presentation | Wireshark tls.handshake                |
| TCP              | Transport                         | 4 Transport                | SYN / SYN-ACK / ACK on port 443        |
| UDP              | Transport                         | 4 Transport                | DNS on port 53                         |
| IPv4             | Internet                          | 3 Network                  | 10.7.x.x addresses, ping               |
| Wi-Fi (802.11)   | Link                              | 2 Data link (+1 Physical)  | MAC addresses in frame headers         |

## 5. Component design

### 5.1 Private DNS (Mac 1, dnsmasq)

**local=/team1.test/** keeps our zone authoritative and never forwarded upstream; every other name goes to 1.1.1.1 / 8.8.8.8, so clients keep internet access.

**host-record** entries map app.team1.test and api.team1.test to 10.7.21.240 with **local-ttl=30**.

Listens only on 127.0.0.1 and 10.7.24.11 (**bind-interfaces**) and logs every query for the live demo.

**.test** is reserved for testing (RFC 6761); **.local** is avoided because macOS uses it for mDNS.

### 5.2 Edge, TLS and load balancing (Mac 2, nginx)

Port 80 returns 301 to HTTPS; port 443 terminates TLS 1.2 / 1.3 with HTTP/2 enabled.

Upstream pool **team1\_backends** uses round-robin across 10.7.22.147:3001 and 10.7.19.69:3002.

Passive health checks: **max\_fails=1 fail\_timeout=10s** removes a dead backend for 10 seconds;

**proxy\_connect\_timeout 2s** and **proxy\_next\_upstream error timeout http\_502 http\_503** retry a failed request on the other backend.

A custom access-log format records the chosen upstream and the X-Backend header for every request.

### 5.3 Certificates

We run our own small CA: **Team1 Local Root CA** (RSA 4096, created on Mac 2) signs the server certificate for app.team1.test with SANs app.team1.test and api.team1.test.

Only the public root certificate is copied to client Macs and set to Always Trust in the System keychain.

Private keys never leave Mac 2 and are excluded from the repository.

Validation is never skipped: curl uses --cacert with the root CA, never -k.

### 5.4 Backends (Mac 3 and Mac 4, Python)

One server.py runs as Backend A (port 3001) or Backend B (port 3002), bound to 0.0.0.0 so Mac 2 can reach it over the LAN.

Every response carries X-Backend: A or B. /api/status uses Cache-Control: no-store so caching never hides load balancing.

/api/info returns identical JSON on both backends with Cache-Control: public, max-age=60 and a matching ETag, so If-None-Match returns 304 Not Modified whichever backend answers.

## 6. Evidence of correct operation

| Task               | What was demonstrated                                                                                                                                         | Evidence              |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| A - LAN            | IP, mask, gateway and MAC of all four Macs; 0% loss ping between them                                                                                         | evidence/01_LAN       |
| B - DNS            | app.team1.test -> 10.7.21.240 from Mac 1, aa flag, TTL 30, clients use 10.7.24.11                                                                             | evidence/02_DNS       |
| C - Backends       | Both backends reachable from Mac 2 by IP with X-Backend headers                                                                                               | evidence/03_BACKENDS  |
| D - Proxy / LB     | Requests alternate A, B, A, B; HTTP/1.1 and HTTP/2 both served; nginx -t passes                                                                               | evidence/04_NGINX     |
| E - TLS            | openssl verify OK, root CA trusted in Keychain, browser and curl show a valid chain, TLS 1.2 and 1.3                                                          | evidence/05_TLS       |
| F - Caching        | Cache-Control + ETag on /api/info; 304 Not Modified from both backends                                                                                        | evidence/06_CACHE     |
| G - Packet capture | DNS, TCP handshake, TLS 1.2 handshake, encrypted application data, ports, flow graph, plain HTTP behind the edge; capture files G1 (TLS 1.2) and G2 (TLS 1.3) | evidence/07_WIRESHARK |

## 7. Failure demonstrations

Each failure was introduced on purpose to show which layer breaks first. The runs use the loopback test setup where each role has its own address: 127.0.0.11 = DNS, 127.0.0.12 = edge, 127.0.0.13 = Backend A, 127.0.0.14 = Backend B.

| #  | Failure                                                        | Last layer that worked                  | First layer that failed                       |
|----|----------------------------------------------------------------|-----------------------------------------|-----------------------------------------------|
| H1 | Client resolver for team1.test pointed at the wrong DNS server | IP: ping to the edge still works        | DNS: no address returned, curl cannot connect |
| H2 | DNS record changed to point at Backend A instead of the edge   | DNS answers, but with the wrong address | TCP: connection refused on port 443           |
| H3 | Backend A stopped                                              | All layers; nginx retries on B          | None for the user: every request served by B  |
| H4 | Both backends stopped                                          | DNS, TCP and TLS to the edge            | Behind the edge: HTTP 502 Bad Gateway         |
| H5 | Client uses port 4443 instead of 443                           | IP and TCP on 443                       | TCP: connection refused on 4443               |

## 8. Known limitations

**Single point of failure** - the nginx edge on Mac 2 is the only entry point. If Mac 2 fails, DNS still works but the service is unreachable. Phase 2 adds a standby edge with DNS-based cutover, using the 30 second TTL.

**Plain HTTP behind the edge** - the edge-to-backend hop is unencrypted inside the LAN (shown in G9). Acceptable for a private LAN; mutual TLS to the backends would remove it.

**DHCP addresses** - the Wi-Fi network assigns addresses dynamically, so the DNS records and nginx upstreams must be updated if a Mac's IP changes.

## 9. Repository

| Path      | Contents                                                         |
|-----------|------------------------------------------------------------------|
| backend/  | server.py and run scripts for Backend A and Backend B            |
| dns/      | team-dnsmasq.conf - DNS configuration for Mac 1                  |
| nginx/    | nginx.conf - edge, TLS and load-balancer configuration for Mac 2 |
| tls/      | certificate-setup.md and openssl-san.cnf                         |
| scripts/  | verify-lan.sh, verify-dns.sh, verify-backends.sh, verify-edge.sh |
| docs/     | architecture.md, this PDF, ip-service-inventory.md, diagrams     |
| evidence/ | Screenshots, logs and packet captures for every task             |

Source: [https://github.com/MananGilhotra/CN\\_Team1](https://github.com/MananGilhotra/CN_Team1)